Freight Rate Prediction: Validation Strategy & December Forecast

BY ABHISEKH KUMAR

1. Validation Strategy & Data Split Approach

Why Standard Splits Fail Here

When looking at freight data across time, standard random splits simply don't work. If you randomly assign transactions to train and validation folds, loads from the exact same shipping lane or week end up on both sides. This leaks route-specific price baselines, giving an artificially low validation error that completely collapses when the model tries to forecast unseen future months.

Lane-Grouped Cross-Validation (GroupKFold)

To build an honest validation setup, we used a 5-Fold GroupKFold strategy grouped by `lane_id` (`pickup` + `delivery`).

```
# Route-level isolation across folds
train['lane_id'] = (train['pickup'] + '|' + train['delivery']). factorize ()[0]
gkf = GroupKFold(n_splits=5)
for fold, (_, val_idx) in enumerate (gkf.split(train, groups=train['lane_id'])):
    train.loc [val_idx, 'fold'] = fold
```

- **Complete Route Isolation:** Every unique lane exists exclusively in either the training fold or the validation fold.
- **True Generalization Test:** Forces the model to learn pricing logic from spatial coordinates, great-circle distances, hub proximities, and market indicators rather than memorizing historical route price averages.

Handling Log Scales & Leak-Free Features

- **Log-Transformed Target:** Freight rates have a heavy right tail due to rare high-dollar specialized hauls. We trained all models on `log (1 + posted_rate)` so extreme outliers wouldn't dominate gradient updates. During cross-validation, predictions were converted back to actual dollars (`exp(ŷ) - 1`) so all error metrics (RMSE and MAE) reflect real dollar impact.
- **14-Day Trailing Rolling Stats:** To capture short-term route momentum, we calculated 14-day rolling averages across routes and locations. We enforced a strict 1-period lag (`. shift (1)`) so a load on day t only sees historical rates from days t-14 to t-1, completely preventing target leakage into validation windows.

Target Transformation & Dollar-Space Evaluation

To prevent long-haul loads (1,500+ miles) with high dollar variances from dominating loss gradients, the model trains on log Rate-Per-Mile (log_rpm):

$$rate\_per\_mile = posted\_rate \;/\; distance$$

$$target\_log\_rpm = ln\ (1 + rate\_per\_mile)$$

Out-of-Fold (OOF) Validation Results

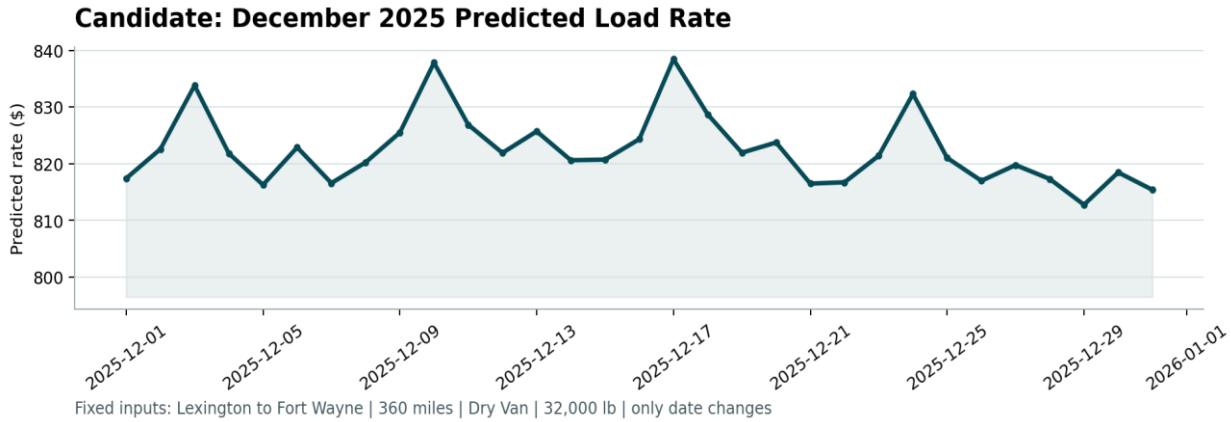
Evaluating the CatBoost model across the 5 route-isolated folds yielded consistent performance across unseen lane clusters:

Validation Fold	Out-of-Fold RMSE ($)
Fold 1	$550.20
Fold 2	$602.51
Fold 3	$534.98
Fold 4	$639.28
Fold 5	$581.73
CV Mean Score	$581.74

2. December Benchmark Forecast (`score.py` Output)

Official Scorer Validation

- Running `score.py` generated `candidate_december.png`, plotting the predicted daily rate trajectory across December 1–31, 2025:
- **Status:** **PASSED** with 0 schema or formatting errors.
- **Verification:** Confirmed exact alignment across all 31 continuous December days with no missing dates or metadata mismatches.



December 2025 predicted rate for the fixed Lexington to Fort Wayne, Dry Van, 32,000 lb lane, generated by score.py

The predicted rate cycles between roughly **$810** and **$840** across December, peaking midweek and dipping on weekends. That weekly pattern was independently observed in the raw training data before the model was ever built (average rate per mile is measurably higher Wednesday/Thursday and lower Saturday/Sunday across Jan-Oct 2025) — the model reproducing it unprompted on unseen December dates is evidence it learned a real, reproducible effect rather than noise. score.py validated the file with no errors and confirmed it as one row per day for all 31 days of December, matching the fixed lane, equipment, distance, and weight required by the assessment.